SOLAR▸SWARM

Bundle

Martin Pham | Perth, AU

Setup & Deployment Guide

This guide walks through setting up the Solar Swarm from scratch on a fresh machine.

Prerequisites

- Python 3.11+
- Node.js 18+ and npm (for the swarm board)
- A terminal and basic familiarity with the command line
- The following API accounts (details below):
- OpenAI (required)
- GoHighLevel (required for production; mock mode works without it)
- Slack (optional — Slack alerts and interactive buttons)
- Retell AI (optional — voice call automation)
- ElevenLabs (optional — premium voice quality)

Step 1: Clone or extract the project

```
# If using git
git clone <your-repo-url> solar-swarm
cd solar-swarm

# If you downloaded a ZIP
unzip Solar-Concept-main.zip
cd Solar-Concept-main
```

Step 2: Create a Python virtual environment

```
python3 -m venv .venv
source .venv/bin/activate   # macOS/Linux
# .venv\Scripts\activate    # Windows
```

Step 3: Install Python dependencies

```
pip install -r requirements.txt
```

This installs:

PACKAGE	PURPOSE
`openai`	GPT-4o for all AI tasks
`flask`	API server
`flask-cors`	CORS for the swarm board
`flask-limiter`	Rate limiting on auth endpoints
`apscheduler`	Scheduled agent jobs
`python-dotenv`	`.env` file loading
`requests`	HTTP calls to GHL, Retell, etc.
`pytz`	Timezone handling
`rich`	Formatted terminal output
`simple-salesforce`	Salesforce CRM support
`slack-sdk`	Slack notifications
`PyJWT`	JWT auth tokens
`bcrypt`	Password hashing

Step 4: Configure environment variables

Copy the example env file:

```
cp .env.example .env
```

Open `.env` and fill in your values:

```
# ── Required ──────────────────────────────────────────────
OPENAI_API_KEY=sk-...           # Get from platform.openai.com

# ── GoHighLevel (primary CRM) ─────────────────────────────
GHL_API_KEY=                    # Settings → Integrations → API → Private Integration Key
GHL_LOCATION_ID=                # Settings → Business Profile → Location ID in URL
GHL_PIPELINE_ID=                # CRM → Pipelines → pipeline ID from URL
GHL_WEBHOOK_SECRET=             # Settings → Integrations → Webhooks → Signing Secret

# ── Slack (optional — for notifications and interactive buttons) ─
SLACK_WEBHOOK_URL=              # Create at api.slack.com/apps → Incoming Webhooks
SLACK_SIGNING_SECRET=           # App credentials → Signing Secret (for /slack/actions)

# ── Voice AI (optional) ───────────────────────────────────
RETELL_API_KEY=                 # app.retellai.com → API Keys
ELEVENLABS_API_KEY=             # elevenlabs.io → Profile → API Key
ELEVENLABS_DEFAULT_VOICE=       # Voice ID from ElevenLabs dashboard

# ── Capital ───────────────────────────────────────────────
WEEKLY_BUDGET_AUD=500           # Total weekly experiment budget

# ── Security ──────────────────────────────────────────────
GATE_API_KEY=                   # Choose any strong random string for the Human Gate API

# ── Ports ─────────────────────────────────────────────────
PORT_HUMAN_GATE=5000
PORT_GHL_WEBHOOKS=5001
PORT_VOICE_WEBHOOK=5002
PORT_DASHBOARD_API=5003

# ── HubSpot (optional fallback CRM) ───────────────────────
HUBSPOT_API_KEY=

# ── Salesforce (optional fallback CRM) ────────────────────
SALESFORCE_USERNAME=
SALESFORCE_PASSWORD=
SALESFORCE_SECURITY_TOKEN=
SALESFORCE_CLIENT_ID=
SALESFORCE_CLIENT_SECRET=
```

The system works with only `OPENAI_API_KEY` set. All other integrations fall back to mock/demo mode when their keys are missing.

Step 5: Initialise the database

```
python3 -c "from memory.database import init_db; init_db()"
```

This creates `swarm.db` in the project root with all tables. The `DATABASE_PATH` env var overrides the default location.

Step 6: Create required directories

```
mkdir -p proposals reports memory/knowledge public
```

DIRECTORY	PURPOSE
`proposals/`	Generated proposal text files
`reports/`	Weekly client report files
`memory/knowledge/`	Warm memory JSON files
`public/`	`board-state.json` for the swarm board

Step 7: Create board-state.json

```
echo '{}' > public/board-state.json
```

Step 8: Test the setup

Run a quick qualification test to verify OpenAI and the database are working:

```
python3 cli.py test-lead
```

Expected output: a qualification result with a score between 1 and 10.

Step 9: Start the system

```
python3 main.py
```

This starts:

- The General scheduler (runs every 6 hours, first run immediately)
- All data collection and research jobs on their scheduled intervals
- Four Flask servers on ports 5000–5003

You should see output like:

```
[DB] Database initialised — swarm.db
[MAIN] Starting Human Gate API on port 5000
[MAIN] Starting GHL Webhook Server on port 5001
[MAIN] Starting Voice Webhook Server on port 5002
[MAIN] Starting Dashboard API on port 5003
[SCHEDULER] All jobs scheduled. Running.
```

Step 10: Verify all services are healthy

```
curl http://localhost:5000/health
curl http://localhost:5001/health
curl http://localhost:5002/voice/health
curl http://localhost:5003/api/health
```

All should return `{"status": "ok", ...}`.

Step 11: Set up the Swarm Board (optional)

```
cd swarm-board
npm install
npm run dev
```

Open `http://localhost:5173` in a browser. The Board tab works immediately (localStorage). The Overview tab shows live data once the Python backend is running.

Step 12: Configure GHL Webhooks (production)

Your server must be publicly accessible for GHL to send events. Use ngrok for development:

```
ngrok http 5001
# Copy the HTTPS URL, e.g. https://abc123.ngrok.io
```

In GHL:

1. Go to **Settings → Integrations → Webhooks → Add Webhook**
2. Set URL to `https://abc123.ngrok.io/webhook/new-lead`
3. Select event: **Contact Created**
4. Add another webhook for **Opportunity Stage Change** → `https://abc123.ngrok.io/webhook/stage-change`
5. Copy the Signing Secret and add to `.env` as `GHL_WEBHOOK_SECRET`

For form submissions:

- Create a GHL Funnel with a form, set webhook URL to `/webhook/form-submit`

For voice call events:

- Point **Conversation Status Changed** to `/webhook/call-complete`

Restart `main.py` after updating `.env`.

Step 13: Set up Voice AI (optional)

```
from voice.retell_client import setup_client_voice_agent

result = setup_client_voice_agent(
    client_id="sunpower_perth",
    company_name="SunPower Perth",
    phone_number="+61892345678",          # E.164 format
    webhook_base_url="https://your-server.com",
    elevenlabs_voice_id="your-voice-id",  # Optional
)
print(result)
# {'success': True, 'agent_id': '...', 'phone': '...', 'phone_linked': True}
```

This creates the Retell agent, links the phone number, and saves the `agent_id` to `company_profiles`.

First Experiment Cycle

After starting `main.py`, the General runs within its first 6-hour window. It will:

1. Generate 3 experiment ideas using GPT-4o
2. Score each with the 4-component confidence system
3. Red-team each idea
4. Route based on adjusted score:

- Score > 8.5 → auto-approved, budget allocated
- Score 5.0–8.5 → Slack alert sent, awaiting your approval
- Score < 5.0 → auto-killed

To trigger the General immediately without waiting:

```
python3 cli.py run-general
```

To see pending experiments:

```
curl -H "Authorization: Bearer $GATE_API_KEY" http://localhost:5000/pending
```

Running in Production

For production deployment, use a process manager to keep `main.py` running:

systemd (Linux):

```
[Unit]
Description=Solar Swarm
After=network.target

[Service]
WorkingDirectory=/opt/solar-swarm
ExecStart=/opt/solar-swarm/.venv/bin/python3 main.py
Restart=always
User=ubuntu
EnvironmentFile=/opt/solar-swarm/.env

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable solar-swarm
sudo systemctl start solar-swarm
```

For the swarm board in production:

```
cd swarm-board && npm run build
# Serve the dist/ folder with nginx or any static host
```

Set `FRONTEND_URL` in `.env` to your deployed frontend URL so the Dashboard API allows CORS from it.

Useful CLI Commands

```
python3 cli.py test-lead              # Test lead qualification
python3 cli.py run-general            # Trigger The General immediately
python3 cli.py run-scout              # Trigger the Scout agent
python3 cli.py status                 # Show swarm summary
python3 cli.py pending                # List pending experiments
python3 cli.py approve <id>           # Approve an experiment
python3 cli.py reject <id> "reason"   # Reject an experiment
```

Solar Swarm — Change Management Process

Version: 1.0

Owner: Martin Pham

Purpose: Define how system changes are evaluated, approved, tested, and deployed

---

Overview

Solar Swarm is a production system handling live client leads and active AI voice calls. Uncontrolled changes can break lead qualification, silently stop pipeline automation, or cause AI calls to fail mid-conversation. This document defines the mandatory process for all changes.

Golden rule: No change to production without a tested rollback path.

Change Categories

Category 1 — Low Risk (No Approval Required)

Changes that cannot affect production lead flow:

- Documentation updates
- Dashboard UI cosmetic changes (colours, layout, labels)
- Adding new Slack notification channels
- Updating `.env` values for non-critical config (LOG_LEVEL, port numbers)
- Adding new monitoring/logging statements
- Updating `board-state.json` content manually

Process: Make change → test locally → commit → deploy

Category 2 — Medium Risk (Self-Approval with Testing)

Changes that affect system behaviour but have quick rollback:

- Adding new API endpoints (non-destructive)
- Changing GPT-4o prompt templates
- Adjusting confidence score thresholds
- Modifying scheduler job intervals
- Adding new database columns (additive only)
- Updating CRM field mappings
- Changing voice call scripts

Process: Test locally → document expected behaviour change → commit with detailed message → deploy → monitor for 30 minutes → confirm or rollback

---

Category 3 — High Risk (Requires Review Checklist)

Changes that affect core business logic or client data:

- Modifying the Kelly Criterion calculator
- Changing circuit breaker thresholds

- Altering database schema (columns removed or renamed)
- Modifying authentication / API key validation
- Changing CRM router logic
- Updating lead qualification scoring algorithm
- Modifying the human gate approval workflow
- Budget allocation changes

Process: Complete change review checklist → test on staging → document rollback steps → deploy with monitoring → confirm or execute rollback

---

Category 4 — Critical (Requires Explicit Sign-Off)

Changes that could cause data loss, billing errors, or security issues:

- Removing database tables or columns
- Changing authentication system
- Modifying circuit breaker halt conditions
- Any change to payment or budget allocation logic
- Changing API keys or secrets rotation process
- Deploying to production during active client calls

Process: Written sign-off in commit message → full staging test → production deploy in maintenance window → immediate monitoring → 24-hour observation period

---

Change Review Checklist (Category 3+)

```
[ ] Change description written in plain English
[ ] Business impact assessed: what could go wrong?
[ ] Affected components identified (list them)
[ ] Database migrations tested locally
[ ] All automated tests pass
[ ] Rollback steps documented (see rollback-procedures.md)
[ ] No hardcoded API keys or secrets introduced
[ ] Logging added for new critical paths
[ ] Dashboard API endpoints return expected responses
[ ] GHL webhook handler tested with sample payload
[ ] Scheduler jobs run without error
[ ] CRM sync completes successfully
[ ] Human gate approve/reject flow verified
```

Deployment Windows

Standard Deployments

- Preferred: Tuesday–Thursday, 10:00–14:00 AWST
- Avoid: Monday mornings (scheduler-heavy), Friday afternoons
- Avoid: During known client peak times (e.g. end of quarter)

Emergency Deployments (Bug Fixes)

- Any time, but notify in #swarm-alerts Slack channel first
- Always have rollback ready before deploying

- Monitor for 15 minutes post-deploy

Blocked Windows (No Deployments)

- During active A/B tests in final 24 hours
- When circuit breaker is at Orange or Red
- When a client onboarding is in progress (first 48 hours)

Version Control Standards

Commit Message Format

```
[TYPE] Short description (under 72 chars)

Body: What changed and why.
Risk level: Low / Medium / High / Critical
Testing: What was tested and how.
Rollback: How to revert if needed.
```

Types:

- `[FIX]` — Bug fix
- `[FEAT]` — New feature
- `[CONFIG]` — Configuration change
- `[SCHEMA]` — Database schema change
- `[PROMPT]` — AI prompt change
- `[SECURITY]` — Security fix
- `[DOCS]` — Documentation only
- `[HOTFIX]` — Emergency production fix

Branch Strategy

```
main          ← production only; auto-deploys
develop       ← integration branch
feature/xxx   ← new features (branch from develop)
fix/xxx       ← bug fixes (branch from main for hotfixes)
```

Incident Response

Severity Levels

P1 — Critical (respond immediately)

- AI calls failing or calling wrong numbers
- Lead qualification returning null scores
- CRM pipeline not updating
- Database corruption or data loss
- Security breach suspected

P2 — High (respond within 1 hour)

- Dashboard API down (board shows stale data)

- Slack notifications not sending
- Scheduler jobs silently failing
- Human gate unreachable

P3 — Medium (respond within 4 hours)

- Individual webhook payloads failing
- Slow API responses (> 5 seconds)
- CRM sync delayed > 2 hours

P4 — Low (next business day)

- Documentation out of date
- Minor UI display issues
- Non-critical log warnings

Incident Process

1. **Detect** — alert from Slack, monitoring, or client report
2. **Contain** — use circuit breaker or disable scheduler job
3. **Diagnose** — check logs, identify root cause
4. **Fix or Rollback** — apply fix or execute rollback procedure
5. **Verify** — confirm system back to normal
6. **Document** — write brief post-mortem (what happened, fix, prevention)

Configuration Change Log

All changes to production `.env` must be recorded:

DATE	KEY CHANGED	CHANGED BY	REASON
YYYY-MM-DD	KEY_NAME	Name	Reason for change

AI Prompt Change Process

Prompt changes are Category 2 (medium risk) but require extra care because:

- GPT-4o outputs can change unpredictably with minor wording shifts
- Qualification score distributions may shift (affecting routing)
- Voice call scripts affect the client's brand and customer experience

Before changing any prompt:

1. Run the existing prompt against 10 test leads — record scores
2. Apply new prompt to same test leads — compare scores
3. Check that hot/cold/borderline classification stays consistent
4. Review AI voice call script changes with a sample call

Change Communication

Internal (Martin / solo operator)

- Document changes in commit messages
- Update CLAUDE.md if architecture changes
- Note prompt changes in `docs/` if significant

Client-Facing

- Notify clients of changes that affect their experience:
- New qualification criteria (discuss first)
- Voice call script changes (send new script for review)
- New pipeline stages or CRM field mappings
- Scheduled maintenance windows (> 30 min downtime)
- Use Slack #solar-clients channel for client notifications

Solar Swarm — Deployment Checklist

Use this checklist for every production deployment.

Pre-Deployment

Environment

- [ ] `.env` file has all required keys (run `python cli.py status`)
- [ ] No placeholder values remaining (e.g. `sk-your-openai-key-here`)
- [ ] Database file exists or will be auto-created (`swarm.db`)
- [ ] Python 3.11+ installed (`python --version`)
- [ ] All dependencies installed (`pip install -r requirements.txt`)

Code

- [ ] All changes committed to git (`git status` clean)
- [ ] Change category identified (see change-management.md)
- [ ] Rollback procedure documented (see rollback-procedures.md)
- [ ] No hardcoded secrets in any file (`grep -r "sk-" . --include="*.py"`)

Testing (complete before every deploy)

- [ ] Import check passes: `python -c "import main; print('OK')"`
- [ ] Config loads: `python -c "import config; print(config.is_configured())"`
- [ ] Database initialises: `python -c "from memory.database import init_db; init_db()"`
- [ ] CRM router loads: `python -c "from integrations.crm_router import status; print(status())"`
- [ ] Dashboard API starts: run `python main.py` and check port 5003 responds

---

Deployment Steps

Step 1 — Stop Current Services

```
# If running as a process
pkill -f "python main.py"

# If running as a systemd service
sudo systemctl stop solar-swarm

# Verify stopped
ps aux | grep main.py
```

Step 2 — Pull Latest Code

```
git pull origin main
```

Step 3 — Update Dependencies

```
pip install -r requirements.txt
```

Step 4 — Run Database Migrations

```
python -c "from memory.database import init_db; init_db(); print('DB OK')"
```

Check for any schema errors in output.

Step 5 — Verify Configuration

```
python cli.py status
```

Expected output: all configured services show green/active status.

Step 6 — Start Services

```
# Development / single-server
python main.py &

# Or as a service
sudo systemctl start solar-swarm
```

Step 7 — Smoke Tests

```
# Dashboard API health
curl http://localhost:5003/api/health

# Human gate health
curl http://localhost:5000/health

# CRM status
curl http://localhost:5003/api/crm/status
```

Expected: all return `{"status": "ok"}` or similar JSON.

Post-Deployment Monitoring (First 30 Minutes)

Check Every 5 Minutes

- [ ] Slack #swarm-alerts — no error messages
- [ ] Dashboard board — live stats updating
- [ ] `tail -f logs/swarm.log` — no exceptions

Verify Scheduler Jobs

After 6 minutes, check that the first scheduler tick ran:

```
grep "General" logs/swarm.log | tail -5
grep "CRM sync" logs/swarm.log | tail -5
```

Verify Lead Flow (If Active Client)

- [ ] Send a test webhook to GHL handler port 5001
- [ ] Confirm qualification score appears in database
- [ ] Confirm Slack alert fires
- [ ] Confirm CRM pipeline stage updated

Deployment Sign-Off

```
Date:           _______________
Deployed by:    _______________
Version/commit: _______________
Change type:    [ ] Cat 1  [ ] Cat 2  [ ] Cat 3  [ ] Cat 4
Issues found:   [ ] None  [ ] Minor (noted below)  [ ] Rolled back
Notes:
_____________________________________________
```

Rollback Trigger Conditions

Execute rollback immediately if:

- Any P1 incident occurs within 30 minutes of deployment
- Lead qualification stops producing scores
- CRM pipeline updates fail
- AI voice calls error > 10% of attempts
- Database integrity errors appear in logs
- Any security alert fires

See `rollback-procedures.md` for rollback steps.

Solar Swarm — Rollback Procedures

Use when a deployment causes a P1 or P2 incident.

Quick Reference

SCENARIO	ACTION	TIME TO RECOVER
Bad code deployed	`git revert` + redeploy	5–10 min
Database migration broke schema	Restore from backup	10–15 min
Wrong `.env` config	Edit `.env` + restart	2 min
Scheduler job looping	Kill job, fix code	5 min
CRM sync corrupted cache	Clear `crm_cache` table	2 min
AI prompt regression	Revert prompt in git	5 min

Procedure 1 — Code Rollback

Use when the deployed code causes errors but the database is intact.

```
# 1. Find the last working commit
git log --oneline -10

# 2. Revert to previous commit (creates a new revert commit)
git revert HEAD --no-edit

# 3. Restart services
pkill -f "python main.py"
python main.py &

# 4. Verify health
curl http://localhost:5003/api/health
curl http://localhost:5000/health
```

If the revert commit approach is too slow, hard reset to previous:

```
# WARNING: This discards the bad commit permanently
git reset --hard HEAD~1
python main.py &
```

Procedure 2 — Database Rollback

Use when a schema migration corrupted the database.

Before any schema migration — always backup first:

```
cp swarm.db swarm.db.backup-$(date +%Y%m%d-%H%M)
```

Restore from backup:

```
# 1. Stop services
pkill -f "python main.py"

# 2. Restore backup
cp swarm.db.backup-YYYYMMDD-HHMM swarm.db

# 3. Verify database
sqlite3 swarm.db ".tables"

# 4. Restart
python main.py &
```

If no backup exists (emergency):

```
# Recreate database from schema (DATA WILL BE LOST — last resort only)
rm swarm.db
python -c "from memory.database import init_db; init_db()"
python main.py &
```

Procedure 3 — Configuration Rollback

Use when a `.env` change caused a misconfiguration.

```
# 1. Open .env and restore previous value
nano .env

# 2. Restart services (no redeploy needed)
pkill -f "python main.py"
python main.py &

# 3. Verify
python cli.py status
```

Procedure 4 — CRM Cache Corruption

Use when CRM sync wrote bad data to `crm_cache`.

```
# 1. Clear the cache table
sqlite3 swarm.db "DELETE FROM crm_cache;"

# 2. Force an immediate CRM sync
python -c "from api.crm_sync import run; run()"

# 3. Verify cache repopulated
sqlite3 swarm.db "SELECT cache_key, cached_at FROM crm_cache LIMIT 10;"
```

Procedure 5 — Scheduler Emergency Stop

Use when a scheduler job is looping, consuming API budget, or stuck.

```
# 1. Kill the main process
pkill -f "python main.py"

# 2. Identify the problematic job in main.py
# Comment out the scheduler.add_job() call for that job

# 3. Restart with the job disabled
python main.py &

# 4. Fix the job code, then re-enable
```

Procedure 6 — Circuit Breaker Reset (Stuck at Red)

Use when the circuit breaker halted the swarm and you've fixed the underlying issue.

```
# Via API (requires GATE_API_KEY)
curl -X POST http://localhost:5000/approve-breaker \
  -H "Authorization: Bearer YOUR_GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'

# Direct database reset (if API is down)
sqlite3 swarm.db "INSERT INTO circuit_breaker_log (level, reason, created_at) VALUES ('green', 'Manual
```

Procedure 7 — AI Voice Call Emergency Disable

Use when AI calls are misfiring or calling wrong numbers.

```
# 1. Set RETELL_API_KEY to empty in .env (disables all outbound calls)
nano .env
# Set: RETELL_API_KEY=

# 2. Restart services
pkill -f "python main.py"
python main.py &

# 3. Verify voice status
curl http://localhost:5003/api/voice/status
# Should return: {"status": "offline"}
```

Post-Rollback Checklist

After any rollback, complete these steps before considering the incident closed:

- [ ] Root cause identified (write 1-2 sentences)
- [ ] All services health-checked and green
- [ ] Test lead processed end-to-end
- [ ] Client notified if they were affected (use plain English, no jargon)
- [ ] Post-mortem added to incident log
- [ ] Preventative measure identified (code fix, test, or process change)
- [ ] Monitoring added to detect the same issue earlier next time

---

Incident Log Template

```
Date:           YYYY-MM-DD HH:MM AWST
Duration:       X minutes
Severity:       P1 / P2 / P3
Trigger:        What caused the incident
Impact:         What stopped working, which clients affected
Detection:      How it was found (alert / client report / monitoring)
Resolution:     Which procedure was used
Root cause:     One sentence explaining why it happened
Prevention:     What change prevents recurrence
```

CRM Integration Guide

Overview: crm_router Abstraction

File: `integrations/crm_router.py`

All agent code calls `crm_router` functions directly. The router selects the active CRM at runtime based on which credentials are configured.

Priority order: GoHighLevel → HubSpot → Salesforce

If GHL_API_KEY is set, all calls go to GHL. If only HubSpot is configured, calls go to HubSpot. Agents never need to know which platform is live.

```
from integrations.crm_router import crm_router

# These calls work identically regardless of which CRM is active
crm_router.get_contact("abc123")
crm_router.create_contact({"name": "Jane Smith", "email": "jane@example.com"})
crm_router.move_pipeline_stage("abc123", "qualified")
```

GoHighLevel (GHL) Setup

GHL is the primary CRM. All clients are expected to be on GHL.

Step 1: Get your API key

1. Log in to your GHL sub-account (not the agency account)
2. Go to **Settings → Integrations → API**
3. Copy the **Private Integration API Key**
4. Add to `.env`: `GHL_API_KEY=your-key-here`

Step 2: Get your Location ID

1. In GHL, go to **Settings → Business Profile**
2. The Location ID is visible in the URL: `app.gohighlevel.com/location/{LOCATION_ID}/...`
3. Add to `.env`: `GHL_LOCATION_ID=your-location-id`

Step 3: Get your Pipeline ID (for stage sync)

1. Go to **CRM → Pipelines**
2. Open the pipeline you want to track
3. The pipeline ID is in the URL
4. Add to `.env`: `GHL_PIPELINE_ID=your-pipeline-id`

Step 4: Configure Webhooks

GHL needs to be configured to send events to the Solar Swarm webhook server.

1. Go to **Settings → Integrations → Webhooks**
2. Click **Add Webhook**
3. Set the URL to your server: `https://your-server.com/webhook/new-lead`
4. Select the following events:

- **Contact Created** → maps to `/webhook/new-lead`

- **Opportunity Stage Change** → maps to `/webhook/stage-change`

1. Copy the **Signing Secret** and add to `.env`: `GHL_WEBHOOK_SECRET=...`

For voice call events:

- Point the **Conversation Status Changed** event to `/webhook/call-complete`

For form submissions:

- Create a GHL Funnel with a form and set the webhook URL to `/webhook/form-submit`

Note: Your server must be publicly accessible. Use ngrok for local development:

```
ngrok http 5001
# Copy the https URL and use it as the base for your webhook URLs
```

GHL Custom Fields Expected

The system reads these custom fields from GHL contact payloads:

FIELD KEY	DESCRIPTION
`homeowner_status`	owner / renter / unknown
`monthly_bill`	Monthly electricity bill in AUD
`roof_type`	tile / colorbond / flat / metal
`roof_age`	Roof age in years

HubSpot Setup

HubSpot is supported as a fallback CRM.

Required credentials

```
HUBSPOT_API_KEY=pat-na1-...  # Private app token (not legacy API key)
```

Getting a Private App Token

1. Go to **Settings → Integrations → Private Apps**
2. Click **Create a private app**
3. Under **Scopes**, enable:

- `crm.objects.contacts.read`
- `crm.objects.contacts.write`
- `crm.objects.deals.read`
- `crm.objects.deals.write`
- `conversations.read`
- `timeline` (for notes)

1. Click **Create app** and copy the token

Salesforce Setup

Salesforce is supported as a third fallback.

Required credentials

```
SALESFORCE_USERNAME=your@email.com
SALESFORCE_PASSWORD=yourpassword
SALESFORCE_SECURITY_TOKEN=yourtoken
SALESFORCE_CLIENT_ID=your-connected-app-client-id
SALESFORCE_CLIENT_SECRET=your-connected-app-secret
```

Getting credentials

1. **Security Token:** Go to **Settings → My Personal Information → Reset My Security Token**
2. **Connected App:** Go to **Setup → Apps → App Manager → New Connected App**

- Enable **OAuth Settings**
- Add scopes: `api`, `refresh_token`
- Save and copy the Consumer Key (Client ID) and Consumer Secret

---

Feature Matrix

FEATURE	GHL	HUBSPOT	SALESFORCE
Get contact	Yes	Yes	Yes
Create contact	Yes	Yes	Yes
Update contact field	Yes	Yes	Yes
Add tag	Yes	Yes (property)	Yes (field)
Move pipeline stage	Yes	Yes (deal stage)	Yes (opportunity stage)
Create task	Yes	Yes	Yes
Send SMS	Yes	No	No
Add note	No (no native API)	Yes	Yes
Find by phone	Yes	Yes	Yes (SOQL)
Get pipeline stages	Yes	Yes	Yes
Inbound webhooks	Yes (native)	Yes (via subscriptions)	Yes (via outbound messages)

How to Switch CRMs

1. Remove or blank out the current CRM's key in `.env`
2. Add the new CRM's credentials
3. Restart `python main.py`

The `active_crm()` function re-evaluates on every call, so no code changes are needed.

To check which CRM is active:

```
curl http://localhost:5003/api/crm/status
```

crm_router Public API Reference

All functions are in `integrations/crm_router.py`. All return `None` on failure (never raise).

FUNCTION	PARAMS	RETURNS	GHL	HUBSPOT	SF
`active_crm()`	—	`str` ('ghl'/'hubspot'/'salesforce'/'none')	—	—	—
`all_configured_crms()`	—	`list[str]`	—	—	—
`is_configured()`	—	`bool`	—	—	—
`status()`	—	`{active, ghl, hubspot, salesforce}`	—	—	—
`get_contact(contact_id)`	`str`	contact dict or None	Yes	Yes	Yes
`create_contact(data)`	`dict`	created contact dict or None	Yes	Yes	Yes
`update_contact_field(contact_id, field, value)`	`str, str, any`	dict or None	Yes	Yes	Yes
`add_contact_tag(contact_id, tag)`	`str, str`	dict or None	Yes	Yes	Yes
`move_pipeline_stage(contact_id, stage_id)`	`str, str`	dict or None	Yes	Yes	Yes
`create_task(contact_id, title, due_date)`	`str, str, str`	dict or None	Yes	Yes	Yes
`send_sms(contact_id, message)`	`str, str`	dict or None	Yes	No	No
`add_note(contact_id, note_body)`	`str, str`	dict or None	No	Yes	Yes
`get_pipeline_stages(pipeline_id=None)`	`str \	None`	list	Yes	Yes
`find_contact_by_phone(phone)`	`str`	dict or None	Yes	Yes	Yes

GHL API Client Reference

The underlying `integrations/ghl_client.py` (used directly when not going through crm_router):

FUNCTION	PARAMS	NOTES
`get_contact(contact_id)`	`str`	GET /contacts/{id}
`update_contact_field(contact_id, field, value)`	`str, str, any`	PUT /contacts/{id} with customField
`move_pipeline_stage(contact_id, stage_id)`	`str, str`	POST /contacts/{id}/pipeline-stage
`add_contact_tag(contact_id, tag)`	`str, str`	POST /contacts/{id}/tags
`create_task(contact_id, title, due_date)`	`str, str, str`	POST /tasks/
`send_sms(contact_id, message)`	`str, str`	POST /conversations/messages (type: SMS)
`create_contact(data)`	`dict`	POST /contacts/ (locationId auto-injected)
`get_pipeline_stages(pipeline_id)`	`str`	GET /opportunities/pipelines/{id}
`is_configured()`	—	True if GHL_API_KEY and GHL_LOCATION_ID both set

All functions return `None` on error. Check `config.GHL_API_KEY` before calling if you want to skip gracefully.

Base URL: `https://services.leadconnectorhq.com`

API Version header: `2021-07-28`

Capital & Risk Management

Kelly Criterion Formula

File: `capital/kelly_engine.py`

The system uses 25% Fractional Kelly to size experiment budgets. This reduces variance while preserving the mathematical edge of full Kelly.

Full Kelly formula

```
f* = (b × p − q) / b

where:
  b = odds (expected ROI ratio, default 3.0 — 3× payoff)
  p = probability of winning
  q = probability of losing = (1 − p)
```

25% Fractional Kelly

```
f_actual = f* × 0.25
budget   = f_actual × WEEKLY_BUDGET_AUD
budget   = min(budget, WEEKLY_BUDGET_AUD × 0.25)  # hard cap: never > 25% per experiment
```

Confidence to Win Probability Mapping

The 4-component confidence score (0–10) is mapped to a win probability before applying Kelly:

```
p = 0.2 + (confidence_score / 10.0 × 0.65)
p = clamp(p, 0.10, 0.90)

# Examples:
# confidence 5.0 → p = 0.20 + 0.325 = 0.525 (52.5% win probability)
# confidence 7.0 → p = 0.20 + 0.455 = 0.655 (65.5%)
# confidence 9.0 → p = 0.20 + 0.585 = 0.785 (78.5%)
```

4-Component Confidence Scoring

```
score_experiment(market_signal, competitive_gap, execution_speed, revenue_path)
# Each component rated 0–10, averaged:
confidence_score = (market_signal + competitive_gap + execution_speed + revenue_path) / 4
```

COMPONENT	WHAT IT MEASURES
`market_signal`	Evidence of real demand for this experiment
`competitive_gap`	How underserved this opportunity is
`execution_speed`	Can this be tested within 72 hours?
`revenue_path`	Is there a clear, direct line to cash?

Example Calculation

```
confidence_score = 7.5
b = 3.0 (default win multiplier)
p = 0.2 + (7.5/10 × 0.65) = 0.6875
q = 1 − 0.6875 = 0.3125
f* = (3.0 × 0.6875 − 0.3125) / 3.0 = 0.5833
f_actual = 0.5833 × 0.25 = 0.1458
budget = 0.1458 × $500 = $72.92
# Hard cap check: $500 × 0.25 = $125 → not triggered
# Budget allocated: $72.92 AUD
```

Budget Buckets

Configured in `config.py`, applied in `capital/portfolio_manager.py`.

BUCKET	ALLOCATION	PURPOSE
`exploit`	60%	Proven strategies; scale what works
`explore`	30%	72-hour test experiments; learn fast
`moonshot`	10%	High-risk, high-reward; long shots only

Bucket assignment rules (`assign_bucket()`):

```
if devil_score >= 7 and confidence_score >= 7:
    return "moonshot"   # Risky but high confidence
elif confidence_score >= 8 and devil_score < 5:
    return "exploit"    # Proven, low risk
else:
    return "explore"    # Default: test it first
```

Checking available budget:

```
get_bucket_budgets()   # {exploit: 300, explore: 150, moonshot: 50, total: 500}
get_bucket_usage()     # Spent this week per bucket
get_bucket_remaining() # Remaining per bucket
can_allocate(bucket, amount)  # True if enough remaining
```

Circuit Breaker States

File: `capital/circuit_breaker.py`

The circuit breaker monitors experiment outcomes and budget burn. It triggers automatic halts before losses compound.

LEVEL	TRIGGER CONDITION	ACTION
Green	All clear	Normal operation
Yellow	3 consecutive experiment failures	Warning logged; system continues
Orange	Budget burn rate > 150% of plan	Warning logged; system slows
Red	5+ consecutive failures OR single loss > 40% of weekly budget	Full halt; General cycle suspended

Threshold values (from `config.py`):

```
CB_YELLOW_FAILURES = 3
CB_ORANGE_BURN_RATE = 1.50      # 150% of planned daily spend
CB_RED_FAILURES = 5
CB_RED_LOSS_FRACTION = 0.40     # 40% of WEEKLY_BUDGET_AUD
```

Checking state:

```
is_halted()                     # True if Red
get_current_level()             # 'green'/'yellow'/'orange'/'red'
get_breaker_history(limit=20)   # Recent events from circuit_breaker_log
```

Resetting a Red breaker:

A Red circuit breaker requires explicit human reset:

```
curl -X POST http://localhost:5000/approve-breaker \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'
```

Or via the Python API:

```
from capital.circuit_breaker import reset_breaker
reset_breaker("martin")
```

Budget burn rate calculation:

```
days_elapsed = max(1, datetime.utcnow().weekday() + 1)  # 1–7
planned_to_date = (WEEKLY_BUDGET_AUD / 7) × days_elapsed
burn_rate = actual_spent / planned_to_date
# burn_rate > 1.5 → Orange trigger
```

Confidence Routing Thresholds

After the 4-component score and red team adjustment:

SCORE	ROUTE	ACTION
> 8.5	`auto_proceed`	Experiment approved immediately, budget allocated
5.0–8.5	`human_gate`	Slack alert sent; human must approve or reject
< 5.0	`auto_kill`	Experiment killed immediately, no budget allocated

Red team downgrade: If `devil_score > 6`, confidence is reduced:

```
penalty = (devil_score - 6) × 0.5
adjusted_confidence = original_confidence - penalty
```

Example: confidence 8.2, devil_score 7 → penalty 0.5 → adjusted 7.7 → human_gate

---

72-Hour Explore Protocol

File: `capital/portfolio_manager.py` — `get_explore_phase()` and `run_explore_monitor()`

All experiments in the `explore` bucket follow a structured 72-hour lifecycle. The `explore_monitor` scheduler job runs every 2 hours to advance phases.

TIME	PHASE	REQUIRED ACTION
0–12h	`asset_creation`	Build landing page, ad copy, creatives
12–24h	`distribution`	Organic channels only — post, email, LinkedIn
24–48h	`signal_observation`	Observe CTR, engagement, enquiries — no spend
48–60h	`decision_point`	If CTR ≥ 2%: activate paid spend; else organic only
60–72h	`final_assessment`	Promote to exploit bucket or kill and log learnings
>72h	`expired`	Auto-killed; mutation engine generates variants on Monday

CTR threshold for paid spend:

```
EXPLORE_CTR_THRESHOLD = 0.02  # 2%
```

If CTR at the decision_point is ≥ 2%, `activate_paid_spend(experiment_id)` is called. This sets `paid_spend_activated=1` on the experiment record and sends a Slack notification.

Phase transitions post Slack notifications automatically when the explore_monitor detects a phase change.

Human Gate Approval Flow

When an experiment is routed to `human_gate` (confidence 5.0–8.5):

1. Experiment saved with `status='pending'` in `experiments` table
2. `alert_human_gate()` posts a Slack message with **Approve** and **Reject** buttons
3. Human clicks button in Slack → Slack sends POST to `/slack/actions`
4. System calls `_approve()` or `_reject()` and updates the message
5. Alternatively, human uses the REST API directly:

```
# Approve
curl -X POST http://localhost:5000/approve/42

# Reject
curl -X POST http://localhost:5000/reject/42 \
  -H "Content-Type: application/json" \
  -d '{"reason": "Budget too tight this week"}'

# See all pending
curl http://localhost:5000/pending
```

On approval: `status` → `approved`, `budget_allocated` calculated, `approved_by` and `approved_at` stamped. Cold ledger entry written with `human_involved=True`.

Pheromone Decay Algorithm

Pheromone signals guide the General toward experiment types that have worked before. Signals degrade over time to prevent stale information from dominating.

Decay parameters:

```
PHEROMONE_DECAY_DAYS = 7     # Signals stay at full strength for 7 days
PHEROMONE_DECAY_RATE = 0.50  # 50% weight loss per decay application
```

Decay formula (applied daily at midnight UTC):

```
new_decay_factor = current_decay_factor × (1.0 − PHEROMONE_DECAY_RATE)
# = current × 0.50

# Signals with decay_factor < 0.01 are treated as expired (invisible to agents)
```

Effect: A signal created 7 days ago has `decay_factor = 1.0`. After the first decay application, `0.5`. After the second, `0.25`. After three days of decay: `0.125`. After ~7 decay days from creation: effectively zero.

Reading active pheromones:

```
from memory.hot_memory import get_active_pheromones
signals = get_active_pheromones(topic="solar_prospecting")  # topic optional
# Returns signals where decay_factor > 0.01, sorted by strength DESC
```

Voice AI Guide

Overview

The voice AI stack enables automated outbound calls to hot leads and inbound call handling for solar company clients. It uses **Retell AI** as the telephony platform and **ElevenLabs** for premium voice quality, with **GPT-4o** as the intelligence layer.

Files:

- `voice/call_handler.py` — Flask webhook server (port 5002)
- `voice/retell_client.py` — Retell AI API client
- `voice/post_call.py` — Post-call transcript processor
- `voice/prompt_templates.py` — System prompt templates
- `voice/call_functions.py` — Function definitions for GPT-4o tool calls

Retell AI Integration

How Calls Are Initiated

Inbound calls:

1. Customer calls the solar company's Retell-managed phone number
2. Retell sends `POST /voice/call-started` to initialise call context
3. For each agent turn, Retell sends `POST /voice/response` with the transcript so far
4. Our server calls GPT-4o with the full system prompt and returns the response
5. When the call ends, Retell sends `POST /voice/post-call` with the full transcript

Outbound calls (hot leads):

1. `qualification_agent` qualifies a lead with score ≥ 7 and phone present
2. Calls `retell_client.create_outbound_call(from_number, to_number, agent_id, metadata)`
3. Retell initiates the call and the same webhook flow applies

Call Flow Diagram

```
Customer ──────────────────── Retell AI Platform
    │                              │
    │ Dials solar company number   │
    │ ──────────────────────────► │
    │                              │
    │                              │ POST /voice/call-started
    │                              │ ──────────────────────────► Solar Swarm
    │                              │                                  │
    │                              │                                  │ Init call context
    │                              │                                  │ Log to call_logs
    │                              │ ◄──────────────────────── │
    │                              │
    │ Speaks...                    │
    │ ──────────────────────────► │
    │                              │ POST /voice/response
    │                              │ (transcript so far)
    │                              │ ──────────────────────────► Solar Swarm
    │                              │                                  │
    │                              │                                  │ Build system prompt
    │                              │                                  │ Call GPT-4o
    │                              │                                  │ Execute function calls
    │                              │ ◄──────────────────────── │
    │                              │   {content: "...", end_call: false}
    │                              │
    │ ◄──────────────────────── │  (AI speaks response)
    │                              │
    │ [Call ends]                  │
    │                              │ POST /voice/post-call
    │                              │ (full transcript + recording)
    │                              │ ──────────────────────────► Solar Swarm
    │                              │                                  │ post_call.py
    │                              │                                  │ Extract → Score → GHL → Slack
```

Setting Up a Client's Voice Agent

Use `retell_client.setup_client_voice_agent()` to do the full setup in one call:

```
from voice.retell_client import setup_client_voice_agent

result = setup_client_voice_agent(
    client_id="sunpower_perth",
    company_name="SunPower Perth",
    phone_number="+61892345678",       # Must be in E.164 format
    webhook_base_url="https://your-server.com",
    elevenlabs_voice_id="your-voice-id",  # Optional
)
# result: {success, agent_id, phone, llm_url, phone_linked}
```

This creates the Retell agent, links the phone number, and saves the `agent_id` to the company profile in the database.

ElevenLabs Voice Selection

ElevenLabs provides higher-quality, more natural-sounding Australian voices than Retell's native TTS.

Default voice: `11labs-Adrian` (configured as `RETELL_DEFAULT_VOICE_ID`)

To use a custom ElevenLabs voice:

1. Create a voice in ElevenLabs dashboard or clone a client's voice
2. Copy the voice ID
3. Pass it to `setup_client_voice_agent(elevenlabs_voice_id="...")`
4. Or set `ELEVENLABS_DEFAULT_VOICE` in `.env` for all new agents

ElevenLabs character cost is tracked per call via `tracking/cost_tracker.py`.

Call Handler Routes

File: `voice/call_handler.py`, served on `PORT_VOICE_WEBHOOK` (default 5002)

METHOD	PATH	DESCRIPTION
GET	`/voice/health`	Health check
POST	`/voice/call-started`	Retell call initialisation
POST	`/voice/response`	Main LLM endpoint — Retell sends transcript, we return next response
POST	`/voice/post-call`	Post-call transcript + recording processing
POST	`/voice/elevenlabs/response`	ElevenLabs alternative to `/voice/response`
POST	`/webhook/email-received`	GHL email forwarding (registered at runtime in `main.py`)

`/voice/call-started` Payload

```
{
  "call_id": "call_abc123",
  "from_number": "+61412345678",
  "to_number": "+61892345678",
  "agent_id": "agent_xyz"
}
```

Initialises `_call_contexts[call_id]` with client_id, phones, and empty lead_data.

`/voice/response` Payload (Retell Custom LLM format)

```
{
  "call_id": "call_abc123",
  "interaction_type": "response_required",
  "response_id": 5,
  "transcript": [
    {"role": "agent", "content": "Hi! Thanks for calling SunPower Perth."},
    {"role": "user", "content": "Hi, I'm interested in solar panels."}
  ]
}
```

Response:

```
{
  "response_id": 5,
  "content": "Great! Let me ask you a few quick questions...",
  "content_complete": true,
  "end_call": false
}
```

`/voice/post-call` Payload

Retell sends the full call data after hang-up:

```
{
  "call_id": "call_abc123",
  "transcript": [...],
  "duration_ms": 180000,
  "recording_url": "https://retell.com/recordings/...",
  "agent_id": "agent_xyz"
}
```

Post-Call Transcript Processing

File: `voice/post_call.py` — `process_post_call(webhook_data, call_ctx)`

Nine steps executed after every call ends:

1. **Extract** — GPT-4o parses the full transcript into structured lead data (name, email, suburb, state, homeowner_status, monthly_bill, roof_type, roof_age, outcome, sentiment, key_objections, follow_up_notes, call_summary)
1. **Merge** — Extracted data merged with in-call context accumulated during the call
1. **Upsert lead** — Creates or updates the `leads` table record
1. **Score** — If not already scored, calls `qualification_agent.qualify()` on the extracted data
1. **Update GHL** — Updates custom fields, adds tags based on outcome, syncs recording URL
1. **Create task** — If `follow_up_required=True`, creates a dated task in GHL for the sales team
1. **Update call_logs** — Stamps duration, recording URL, outcome, score on the `call_logs` record
1. **Slack notification** — Posts call outcome with score emoji and summary
1. **Cold ledger** — Writes `VOICE_CALL_COMPLETE` event

GHL tags applied by outcome:

OUTCOME	TAGS ADDED
`booked_assessment`	assessment-booked, voice-ai-qualified
`callback_requested`	callback-requested, voice-ai-lead
`not_interested`	not-interested-voice
`transferred`	transferred-to-human
Score ≥ 7	hot-lead
Battery interest detected	battery-interest
EV owner detected	ev-owner

Prompt Templates

File: `voice/prompt_templates.py`

The system supports multiple prompt templates for different call scenarios. The template is selected via `_build_system_prompt(client_id, call_id, template)`.

TEMPLATE NAME	USE CASE
`inbound_solar`	Default; inbound lead qualification
`outbound_cold`	Cold outreach to scraped prospects
`outbound_callback`	Hot lead callback (score ≥ 7)
`support`	Existing customer support

Template variables injected:

VARIABLE	SOURCE
`{company_name}`	`company_profiles` table or DEFAULT_CLIENT_ID
`{company_knowledge}`	`knowledge/company_kb.py` — products, pricing, rebates
`{today}`	Current date
`{call_id}`	From Retell

Voice Rules Enforced by System Prompt

- Keep responses under 3 sentences
- No bullet points — weave into natural speech
- Australian English (colour, neighbourhood, organise, mum)
- Never say "as an AI" unless directly asked
- Call `lookup_caller` at the very start of every call
- Call `qualify_and_score` once homeowner status, bill, and roof info are collected
- Call `end_call` when the conversation is naturally complete

How to Add a New Voice Prompt Template

1. Open `voice/prompt_templates.py`
2. Add a new key to the `TEMPLATES` dict with your prompt string
3. Include the same `{company_name}`, `{company_knowledge}`, `{today}`, `{call_id}` placeholders
4. Call `_build_system_prompt(client_id, call_id, template="your_new_template")` in the handler

Cost Tracking Per Call

Every call has its costs tracked in the `api_usage` table via `tracking/cost_tracker.py`:

Retell AI cost: Logged in `voice/call_handler.py` post-call handler

```
log_retell_call(
    call_id=call_id,
    duration_seconds=duration_s,
    client_id=ctx.get("client_id"),
    agent_id=data.get("agent_id"),
)
```

OpenAI tokens: Logged on every GPT-4o call inside `_call_llm()`:

```
log_openai(
    model=config.OPENAI_MODEL,
    prompt_tokens=usage.prompt_tokens,
    completion_tokens=usage.completion_tokens,
)
```

To see cost for a specific call:

```
curl "http://localhost:5000/costs?call_id=call_abc123"
```

Retell pricing (approximate): ~$0.02–$0.05 USD per minute depending on plan.

ElevenLabs pricing: Per character of generated speech; typical call uses ~500–2000 characters.

GPT-4o pricing: ~8–20 API calls per call; ~$0.01–$0.05 per call depending on length.